

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI
K K BIRLA GOA CAMPUS

2025-26 Sem. I	ComprExam Regular	Open Book
Principles of Programming Languages	CS F301	35%=70 marks; Pages=3; Questions=4.viii
6 Dec 2025	180 min	1400-1700 hrs
If you choose clumsy handwriting and strike off partial text you are forfeiting recheck privileges. Clarity, consistency and precision are important.		

Q1. For each of the software use cases below, and the design goals provided with them, choose a paradigm and one or more programming languages that you'd consider as the best combination for the use-case design-goal combination, with reason. Choose the paradigms from amongst Procedural (Modular), Modular-OO (equivalent to Pure OO), Pure Functional, Functional-OO (equivalent to OO-Functional). Choose languages from amongst Fortran, Lisp, C, C++, Java, Python. [Marks (2+2+2)×3=18]

- (i) **Flight Management Systems (FMS):** the “brains” of the aircraft, optimizing flight paths, saving fuel, and providing navigation assistance to pilots.
Design Goals: Computation speed, optimization accuracy.
- (ii) **Autopilot Systems:** to handle complex maneuvers, from altitude hold to automated landings, allowing pilots to focus on strategic decision-making.
Design Goals: Mission critical guarantees on accuracy, timing and safety.
- (iii) **Maintenance & Diagnostics Software:** for predictive maintenance using sensor data for
Design Goals: forecasting potential component failures, reducing downtime and ensuring regulatory compliance.

Q2. Did you implement your ENDSEM assignment at the top level as an REPL (Python-like READ-EVAL-PRINT LOOP) or an event-loop (a reactive system that responds to menu-mouse-keypad-touchscreen controls, like a smartphone or a desktop manager or a docker or a browser)? If yes, explain why. If not, explain what did you implement in its place and why. In either case, categorise your thinking and design process as one of the four: procedural, OO, pure functional, functional-OO. Justify why you use that category to label it with. Your answer needs to be consistent with your submission, and the overall length not more than 5 to 10 lines. Irrelevant verbose answers, clumsy and inconsistent answers will nix your chances of partial marks and recheck privileges. Write anything only if you are sure to really stand by the whole of it even after the exam. [Marks 5+2+5=12]

Q3. Study the following implementation of QUICKSORT in Python.

```
1 def quick_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     pivot = arr[0]
5     left = [x for x in arr[1:] if x < pivot]
6     right = [x for x in arr[1:] if x >= pivot]
7     return quick_sort(left) + [pivot] + quick_sort(right)
```

The Question: Compare this implementation with the usual C, C++, Java implementations using the ordinary **for** loops, with justification **in no more than 1 line** each in terms of [Marks (1+3)x5=20]

- (i) Run-time efficiency
- (ii) Space efficiency (use of extra storage apart from **arr** itself).
- (iii) Versatility (type flexibility: will the code just work without changes or copies for an input array of any element type?)
- (iv) Sorting stability (will two “equal” elements—like the comparison is on keys between records that have identical keys but different payload data—appear in the same sequence in the sorted output as their original sequence?)
- (v) Suitability for sorting a terrabyte long online stream.

Remember: the comparison is between only **two** implementations, mainly on the special Python **for** loop in lines 5,6 above against the usual **for** loop.

Assume Lomuto partitioning for the C, C++, Java implementations: both the indices during partitioning move left to right, unlike Hoare partitioning in which they move in opposite directions from each end towards the middle.

Q4. Draw the procedure activation records of the complete trace of the call **fibDP(5)** to the following C function: [Marks 20]

```
int fiboDP(int n) {
    static int fib[100];
    if(n > 1 && n < 100) {
        if(fib[n] == 0 {
            if(fib[n-2] == 0) fib[n-2] = fiboDP(n-2);
            if(fib[n-1] == 0) fib[n-1] = fiboDP(n-1);
            fib[n] = fib[n-2] + fib[n-1];
        }
        return fib[n];
    }
    else return 1;
}
```

SOLUTION

Answer1.

Use Case	Paradigm	Programming Language	Reason
(i)✈	Modular OO	C++, Fortran	Both the languages offer reliable systems control, computation efficiency and interoperability to byte-level and clock-cycle level precision.
(ii)✈	Modular OO	C++, Fortran	The same considerations as (i). (<i>Note: In practice, Ada was designed and remained dominant in usage for this purpose. However, to avoid whimpering attacks on me for “out of syllabus” questions I did not add Ada to the list of languages.</i>)
(iii)✂	Functional-OO	All the 5 languages in the list.	This takes real-time data and makes real-time predictions but nothing is mission-critical, so availability of system software and programmers’ skills are the criteria.

For TAs: The paradigms and languages are fixed, no partial marking in that. In the reasoning, for the first two, control is an important keyword. Students may compare these use cases with other mission-critical use cases; that is valid reasoning. For the third, the main point is about programmers’ convenience. Students may compare this with other interactive business operations software use cases, like banking and airline reservations. That is also a valid answer.

Answer2. Since this is open-ended, consistency with your submission is very important. Very few proper REPL or event-loop systems I have seen among the submissions. At the best the above-average submissions have given a CLI that repeatedly seeks one of the few application options and provides some data aggregation as a result. I didn’t expect a Jupyter notebook level thing from anyone but at least some inspiration drawn from it would be enough for a sense of contentment.

The main reasons why this is so (*that most students do not think of designing and building reactive systems; they still write one integral program that seeks to do “exactly everything that is asked for by the examiner and nothing more or less”*) are, IMHO:

- Predominance of procedural thinking habits structured and reinforced by our lab test components for most of the courses.
- The glamour of competitive coding.
- Lack of project experience (or lack of design challenges and long-term vision in the projects that you do)
- Lack of patience and conceptual clarity for conscious design before implementation.

The difference between an REPL and an event-loop: The former is a CLI shell like Python (or GNU Octave, Lisp, Jupyter notebook, etc.). The latter is a GUI with windows, menus, buttons, switches, pop-ups, textbox inputs, etc. For example, one could provide an asynchronous interface with input and keyword files independently read and changed, merged, extracted, etc.

Note for TAs: division of 5+2+5:

1. **Identification** If students give a clear answer (their implementation being REPL/event loop/none) **consistent** with their submission for the endsem assignment, give 5 marks. Otherwise 0. Partial marks should be only sparingly given, and are subjective and non-negotiable during rechecks.
2. **Justification** If they had identified their top level interface and justified it in their report then (perhaps without the particular terms REPL and/or event-loop, but a meaningful precise description) and repeat that in this answer consistently with one of these particular terms, give 2; if there was none in the report but given here that is consistent with their submission, give 1; otherwise 0.
3. **Paradigm** An REPL is either Pure Functional or (more likely) Functional OO. An event loop is Functional-OO. In both the cases, computation proceeds asynchronously (or in a stimulus-response reactive manner) not procedurally. Any other top-level interface, especially the more common text-menu-driven CLI that most students provided, is Modular-OO or just Procedural. In this case, computation proceeds via a single interactive thread with the startup main is always in control and the user guides each step.

Answer3. The main difference between the two kinds of implementations is the interaction of the logical operation of partitioning with the storage structure operations. The Python implementation is **not in place**, it creates a new part each for the left and the right parts of the partition by a map-filter, and then concatenates them into a **new** list. The usual C, C++, Java implementations perform **in place swaps** partitioning **the same array without copying**.

- (i) Surely the usual non-Python implementations are all more efficient because they depend on indexed access to arrays and no unnecessary copying (only necessary swaps).
- (ii) Similarly, the usual non-Python implementations all use constant extra space (1 temporary element for swaps), thus more space-efficient.
- (iii) Of course Python is more versatile, type-flexible, since it is dynamically typed.
- (iv) Assuming that the implementation of the map-filter for loop in Python scans the array left-to-right and copies into the new array in the same order, it is stable. The usual Lomuto-partitioning is not stable inherently. (Hoare partitioning too.)
- (v) The given Python implementation is not suitable for any large array even if all of it fits into the main memory at once. Because it doesn't do in-place sorting. It needs two copies of the array in each iteration. The usual implementations are suitable for this. In fact that is why quicksort implemented in C-style is the most common algorithm used for any infrastructure sort utility on all platforms, or embedded in large software packages like spreadsheet editors.

Answer4. The call-graph with abbreviated Proc.Act.Rec.s is shown:

5CR	3CR	1CR	
5CR	3CR	2CR	0CR
5CR	3CR	2CR	1CR
5CR	4CR	2CR	
5CR	4CR	3CR	

Note to TAs: If students show space for a return value and a temporary variable for the sum (after CR) on the stack in Answer 4, give them credit too. Not extra, not less. What is important is showing 9 calls with max frames at a time 4. 17 total frames to be shown, 3 marks each. Give 21 to someone who really did all correctly, no problem. Unmatched call graphs will get less, at your discretion.